0

OPPORTIX

Plateforme de gestion de flotte
Documentation de la logique métier

Document technique · Avril 2026
Périmètre : URBAN NEO & GREEN

Sommaire

1. Vue d'ensemble & architecture
2. Modèle de données
3. États d'une voiture
4. Cas par cas — Départs
5. Cas par cas — Retours
6. Cas par cas — Dispo
7. Cas par cas — Garage
8. Cas par cas — Vacances chauffeurs
9. Cas par cas — Départs & Retours à prévoir
10. Auto-réparations au chargement
11. Historique conducteurs
12. Archivage
13. Sauvegarde & synchronisation temps réel
14. Rôles et permissions

1 — Vue d'ensemble & architecture

OPPORTIX est une application web qui gère deux flottes parallèles de véhicules : `URBAN NEO` et `GREEN`. Elle suit en temps réel les départs, les retours, les voitures disponibles, celles au garage, et les vacances des chauffeurs.

L'application est composée de trois briques :

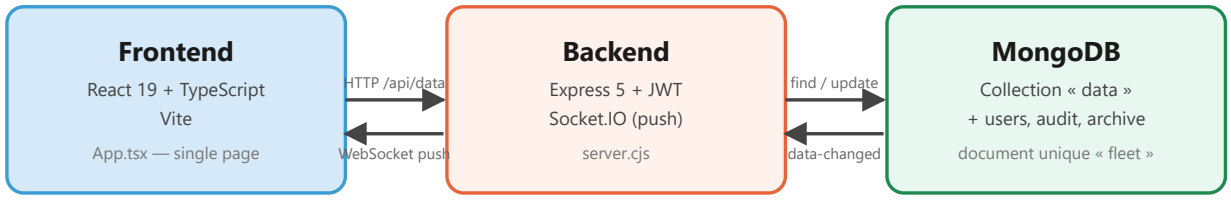

Figure 1 — Architecture trois tiers. Le frontend persiste tout via PUT /api/data, le serveur diffuse les changements via WebSocket à tous les clients connectés.

Toute la logique métier (transitions d'état, sync entre listes, annulations automatiques de vacances, etc.) vit côté **frontend**, dans `src/App.tsx`. Le backend est un thin layer : il valide les clés, persiste, diffuse, et tient un journal d'audit + une corbeille.

2 — Modèle de données

Toutes les données opérationnelles vivent dans **un seul document MongoDB** (clé `_key: 'fleet'`). Ce document contient les sous-tableaux suivants :

Clé	Contenu	Affiché dans
`u`	Voitures de la flotte URBAN NEO	Onglets Véhicules / Flotte
`g`	Voitures de la flotte GREEN	Onglets Véhicules / Flotte
`dep`	Départs (assignations actives)	Onglet Départs
`ret`	Retours	Onglet Retours
`di`	Voitures en dispo (pool de disponibilité)	Diffusion + onglet VH Dispo
`ga`	Voitures au garage	Diffusion (split urb/grn) + onglet Garage
`va`	Vacances chauffeurs	Diffusion + onglet Vacances
`dpv`	Départs à prévoir (brouillons commerciaux)	Diffusion
`rpv`	Retours à prévoir (brouillons)	Diffusion
`msgs`	Messages internes inter-pôles	Onglet Messagerie

Forme d'une voiture (flotte u / g)

`{ im, mq, mo, le, st, ch }` → immatriculation, marque, modèle, leaser, statut, chauffeur.

Forme d'un départ

`{ id, soc, im, mo, le, ch, dt, no }` → société, immat, modèle, leaser, chauffeur, date, note.

Forme d'un départ à prévoir

`{ id, soc, im?, mo?, le?, ch?, dt?, no?, co? }` — tous les champs sont optionnels (sauf `id`). En plus du départ classique, il y a un champ `co` (commentaire libre).

> **À retenir :** il n'y a pas de jointure ni de relation entre tables. C'est un seul document JSON sauvegardé d'un bloc à chaque modification. Toute la cohérence (par exemple "une voiture en mission ne peut pas être en dispo") est garantie par la logique frontend.

3 — États d'une voiture

Une voiture appartient toujours à exactement une flotte (urban OU green) et a toujours un état combinant `st` et `ch` :

Statut	Chauffeur	Signification
`ACTIF`	nom du chauffeur	En mission, livraison en cours
`IMMO`	`BUREAU`	Au repos, disponible
`IMMO`	`VACANCES`	Conducteur en congé
`IMMO`	`GARAGE [nom]`	En réparation

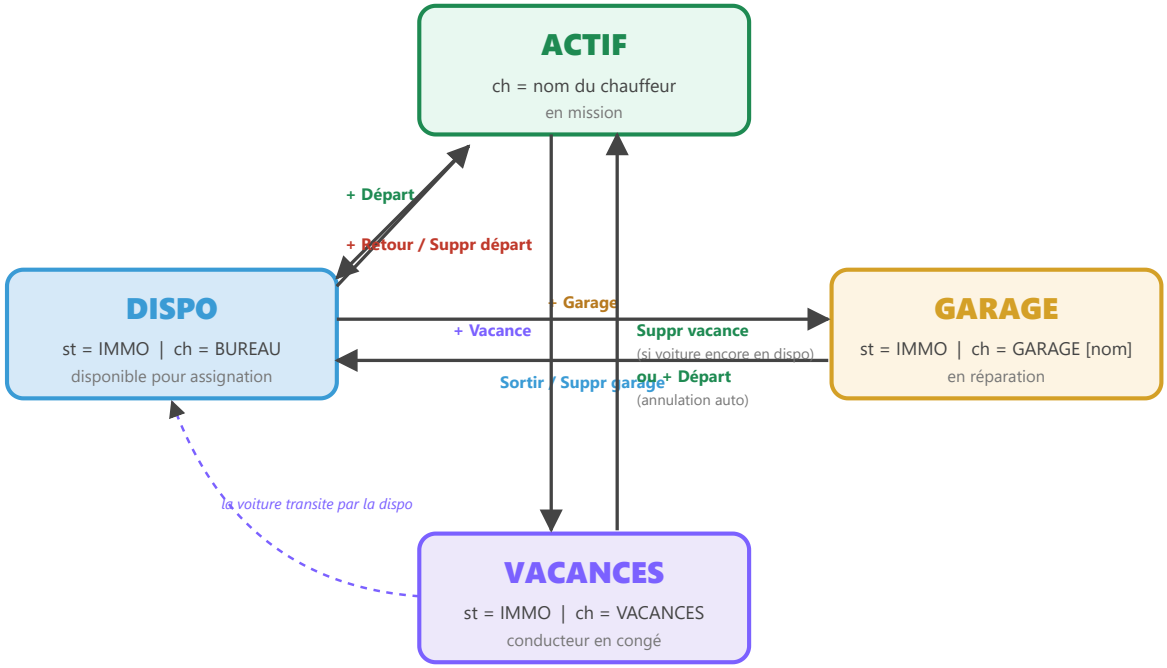

Figure 2 — Machine à états d'une voiture. Les transitions sont déclenchées par les actions utilisateur sur les listes (départ, retour, garage, vacance).

4 — Cas par cas : Départs

4.1 — Ajouter un départ

C'est le cas le plus riche de l'application. Une voiture qui part en mission déclenche cinq sous-actions automatiques :

1. **Sortie de dispo** — si l'immat se trouvait dans le pool dispo, elle en est retirée.
2. **Sortie du garage** — si elle y était, elle en sort aussi.
3. **Création flotte si manquante** — si l'immat n'existe ni en URBAN NEO ni en GREEN, la voiture est **créée** dans la bonne flotte (selon la `soc` du départ).
4. **Synchronisation flotte** — la voiture passe à `ACTIF` avec le chauffeur du départ. Marque, modèle et leaser sont remplis depuis le départ s'ils manquaient.
5. **Annulation vacances auto** — si le chauffeur avait une vacance enregistrée, elle est **supprimée et archivée** automatiquement. Plus besoin de la retirer manuellement.

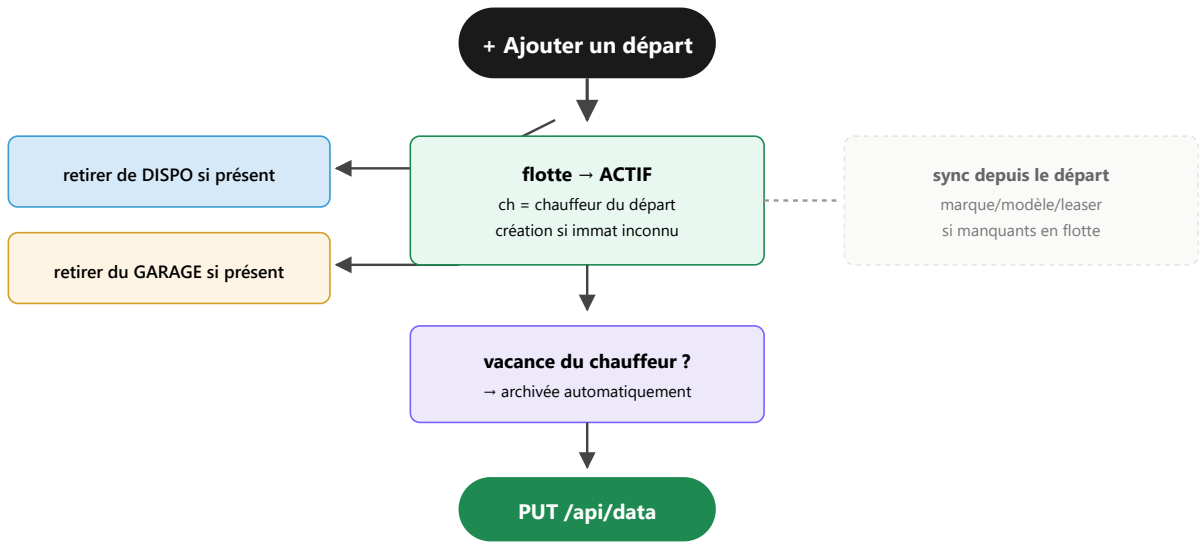

Figure 3 — Workflow d'ajout d'un départ. Toutes les étapes sont packagées dans une seule sauvegarde (PUT /api/data), pas d'état intermédiaire visible.

4.2 — Éditer un départ

- Met à jour les champs (modèle, leaser, chauffeur, date, note) dans la liste des départs.
- **Si la société change** (URBAN NEO ↔ GREEN), la voiture est **physiquement déplacée** d'une flotte à l'autre, en gardant tous ses attributs.
- Le modèle et le leaser sont aussi propagés vers la fiche flotte.

4.3 — Supprimer un départ

1. Le départ est archivé dans la collection `archive`.
2. La voiture **revient automatiquement en dispo** avec `st: IMMO`, `ch: BUREAU`.

> Suppression = annulation. C'est utilisé quand un départ a été créé par erreur, ou quand un livreur a finalement renoncé à la mission. Si tu veux marquer un départ comme terminé, c'est un **retour** qu'il faut créer.

5 — Cas par cas : Retours

5.1 — Ajouter un retour

1. Si la voiture n'est pas déjà en dispo, elle y est ajoutée (avec son modèle récupéré depuis la flotte).
2. La voiture passe à `IMMO`, `ch: BUREAU` dans la flotte.

5.2 — Supprimer un retour

Retire simplement la voiture de la dispo. La flotte n'est pas touchée.

6 — Cas par cas : Dispo

La dispo est le **pool de voitures disponibles**. C'est l'état "neutre" : la voiture est physiquement là, prête à être réassignée.

6.1 — Ajouter manuellement à la dispo

1. Création de la voiture dans la flotte si elle n'existe pas encore (selon la `soc` du formulaire).
2. Forçage à `IMMO`, `ch: BUREAU`.

6.2 — Comment une voiture arrive en dispo automatiquement

- Suppression d'un départ → retour en dispo
- Ajout d'un retour → entrée en dispo
- Ajout d'une vacance → la voiture du conducteur passe en dispo (note "VACANCES")
- Sortie ou suppression d'une fiche garage → entrée en dispo (note "Sortie garage [nom]")

6.3 — Supprimer une voiture de la dispo

Simple retrait de la liste. Aucun effet de bord sur la flotte (la voiture reste en IMMO BUREAU jusqu'à ce qu'un autre événement la modifie).

7 — Cas par cas : Garage

Sur la page diffusion, le garage est **séparé en deux blocs** : `GARAGE URBAN` et `GARAGE GREEN`. La société est implicite par bloc, donc le formulaire d'ajout n'a plus à la demander.

7.1 — Ajouter au garage

1. La voiture passe à `IMMO` avec `ch: GARAGE [nom du garage]`.
2. Elle est retirée de la dispo si elle y était.
3. **Réconciliation flotte** — si la voiture est dans la "mauvaise" flotte (par exemple en URBAN NEO mais on l'ajoute au GARAGE GREEN), elle est **déplacée** vers la flotte cible. La société du garage est la source de vérité.
4. Si l'immat n'existe nulle part, la voiture est créée dans la bonne flotte.

Figure 4 — Cycle garage. Le bouton « Sortir » (orange) déplace vers dispo. Le bouton « × Retirer » (rouge) fait la même chose mais archive en plus la ligne (utilisé pour corriger une erreur de saisie).

7.2 — Sortir du garage (bouton orange « Sortir »)

1. La voiture est retirée du garage.
2. Elle est ajoutée en dispo avec une note `Sortie garage [nom]`.
3. La flotte garde son statut `IMMO` (elle n'est pas réassignée tant qu'on ne crée pas un départ).

7.3 — Supprimer une fiche garage (bouton ×)

Effet identique au bouton « Sortir », mais en plus la fiche est archivée. C'est destiné à corriger une erreur de saisie.

8 — Cas par cas : Vacances chauffeurs

8.1 — Ajouter une vacance

1. L'app cherche la voiture actuellement attribuée au chauffeur (par `ch` dans la flotte).
2. Si elle existe et n'est pas déjà en dispo, elle est ajoutée à la dispo avec note `VACANCES JJ/MM-JJ/MM`.
3. La voiture passe à `IMMO`, `ch: VACANCES`.

8.2 — Supprimer une vacance — deux scénarios

Quand une vacance est supprimée, le chauffeur est libéré mais on ne le réattribue **pas automatiquement** à son ancienne voiture. Dans la pratique, un chauffeur qui revient de congé reçoit fréquemment une voiture différente — le lien chauffeur ↔ voiture est porté par le prochain départ qu'on lui crée, pas par sa précédente attribution.

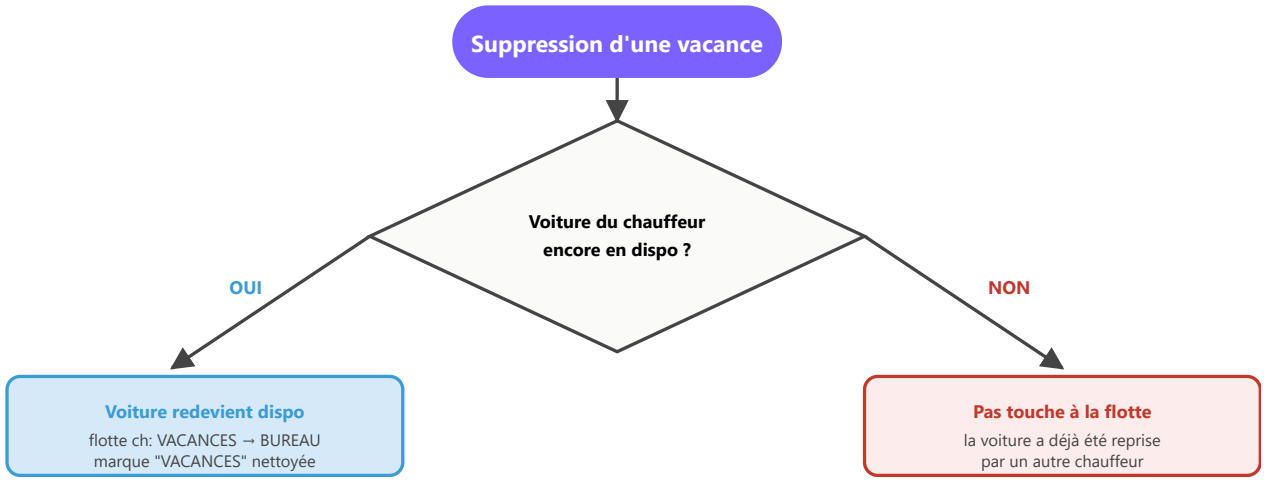

Figure 5 — Suppression d'une vacance. Dans tous les cas, le chauffeur n'est pas réattribué à son ancienne voiture — il sera lié à une nouvelle voiture (peut-être la même, peut-être une autre) lors de son prochain départ.

Cas 1 — la voiture est encore dans le pool dispo (avec une note "VACANCES") : on enlève la marque VACANCES côté dispo et côté flotte (la fiche passe de `ch: VACANCES` à `ch: BUREAU`). La voiture devient une voiture dispo standard, prête à être attribuée à n'importe qui.

Cas 2 — la voiture a déjà été reprise par quelqu'un d'autre entre temps (via un départ) : on ne touche à rien d'autre que la liste des vacances. La voiture est déjà dans son nouvel état correct.

8.3 — Annulation automatique par un départ

Si tu crées un départ pour un chauffeur qui est **en vacances**, sa vacance est **automatiquement archivée** et supprimée. Le chauffeur passe immédiatement en mission (cf. section 4.1, étape 5).

9 — Cas par cas : Départs et Retours à prévoir

Cette section est pensée pour les **commerciaux**. Quand on sait qu'une mission va arriver mais qu'on ne connaît pas encore tous les détails (immat, chauffeur, date exacte), on peut quand même créer une **fiche brouillon**.

9.1 — Ajouter un départ à prévoir

- **Aucun champ n'est obligatoire.** Tu peux créer une ligne avec juste un commentaire ("Livraison Bordeaux semaine pro").
- Aucun effet de bord : la flotte n'est pas modifiée, aucune voiture n'est déplacée.
- La fiche apparaît dans le bloc « DEPARTS À PRÉVOIR » de la diffusion.

9.2 — Compléter une fiche

Le commercial peut éditer la fiche au fil de l'eau (immat trouvée, chauffeur attribué, date confirmée…). Champs : société, immat, modèle, leaser, chauffeur, date, note, **commentaire libre**.

9.3 — Confirmer (bouton ✓ vert)

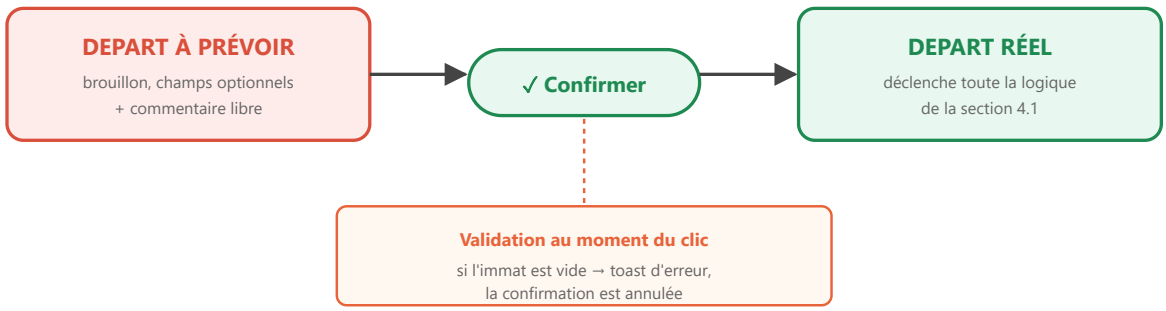

Figure 6 — Flow de confirmation d'un départ à prévoir. La note et le commentaire sont fusionnés au format « note | commentaire » avant d'être passés à la fonction add() classique.

1. **Validation** : si l'immat est vide → toast d'erreur, on n'avance pas.
2. La note et le commentaire sont fusionnés au format `note | commentaire`.
3. La fonction d'ajout de départ classique est appelée — donc **tous les effets de bord de la section 4.1 s'enclenchent** : sortie de dispo, sortie de garage, annulation de vacances, création flotte si manquante, etc.

4. La fiche disparaît de la liste « à prévoir » et apparaît dans l'onglet Départs.

9.4 — Supprimer une fiche à prévoir

Retire simplement la ligne, sans rien d'autre. C'est juste un brouillon qu'on jette.

Le mécanisme est strictement le même pour les **retours à prévoir**.

10 — Auto-réparations au chargement

À chaque ouverture de page (et toutes les 15 secondes en fallback), l'app récupère le document complet via `GET /api/data`, puis applique cinq passes de réparation automatique avant d'afficher les données :

1. **Re-parse marque/modèle** — si une voiture a `mo: "BYD SEAL"` mais `mq` vide, c'est splitté en `mq: "BYD"`, `mo: "SEAL"`.
2. **Sync depuis les départs** — pour chaque départ, si l'immat n'existe pas dans la flotte, la voiture est créée dans la bonne société. Si elle existe mais sans modèle/leaser, ils sont remplis depuis le départ.
3. **Patch ids garage** — tout item garage sans `id` reçoit un UUID en mémoire (nécessaire pour que les boutons Sortir / Retirer fonctionnent uniformément).
4. **Patch société garage** — tout item garage sans `soc` est classé URBAN ou GREEN selon dans quelle flotte se trouve son immat. Comme ça, aucune voiture ne « disparaît » dans le split urb/grn.
5. **Persistance** — si l'une de ces réparations a modifié quelque chose, les changements sont immédiatement sauvegardés via PUT /api/data.

> Cette logique d'auto-réparation rend l'app résistante aux imports manuels, aux anciennes données, et aux corruptions partielles. Elle ne fait jamais disparaître de données — elle ne fait qu'enrichir.

11 — Historique conducteurs

Endpoint : `GET /api/history`. À chaque appel, le serveur :

1. Récupère les départs et retours **actuels** (depuis le document fleet).
2. Récupère les départs et retours **archivés** (collection `archive`).
3. Fusionne les deux et groupe par chauffeur (nom en majuscules, espaces trim).
4. Ajoute aussi le véhicule actuellement attribué dans la flotte (entrée de type `vehicule_actuel`).

Le résultat est un objet `{ NOM_CHAUFFEUR: [événements...] }`, affiché dans l'onglet Historique avec une timeline triée par chauffeur. Comme ça, on peut tout savoir d'un livreur : quelle voiture il a actuellement, ses derniers départs, ses derniers retours, même ceux qu'on a supprimés / archivés.

12 — Archivage (suppression douce)

Toute « suppression » d'un item (départ, retour, vacance, garage, dispo, à-prévoir, etc.) passe par `POST /api/archive` qui stocke l'item dans la collection `archive` avec :

Champ	Contenu
`section`	type d'item (deps, rets, vacs, garage, ...)
`item`	la donnée complète au moment de la suppression
`deletedBy`	username de l'utilisateur
`deletedAt`	timestamp

Rien n'est jamais réellement perdu côté base. L'admin peut consulter les 200 dernières suppressions via `GET /api/archive`.

13 — Sauvegarde & synchronisation temps réel

13.1 — Sauvegarde

Toutes les écritures passent par la même fonction frontend, qui PUT le document complet sur `/api/data`. Côté serveur, seules les clés suivantes sont autorisées : `u, g, ga, dep, ret, di, va, pr, dpv, rpv, msgs`. Toute autre clé fait échouer la requête.

13.2 — Verrou anti-race

Un drapeau `savingRef` empêche un rechargement de tomber pendant une sauvegarde en cours, et inversement. Sans ça, on aurait des écrasements involontaires quand plusieurs onglets sont ouverts.

13.3 — Push temps réel

Figure 7 — Push temps réel via Socket.IO. Quand un utilisateur sauvegarde, tous les autres clients connectés (room « authed ») reçoivent l'événement et rechargent automatiquement les données.

13.4 — Fallback polling

Au cas où le WebSocket sauterait (réseau, proxy…), chaque client refait un GET /api/data toutes les 15 secondes pour rester à jour. C'est invisible pour l'utilisateur.

14 — Rôles et permissions

Rôle	Lecture	Modifications	Gestion utilisateurs	Audit / Archive
lecteur	✓	✗	✗	✗
editeur	✓	✓	✗	✗
admin	✓	✓	✓	✓

Côté serveur, trois middlewares font respecter les règles : `canEdit` (bloque les lecteurs), `canManageUsers` (bloque les lecteurs) et `adminOnly` (admin uniquement). Côté frontend, tous les boutons d'action sont conditionnés sur `user.role !== 'lecteur'`, donc un lecteur ne voit même pas les boutons d'ajout/édition/suppression.

Authentification par **JWT**, expiration 12 heures, mots de passe stockés en bcrypt. Login limité à 10 tentatives par 15 minutes. API limitée à 120 requêtes par minute.

— Fin du document —